Assertion

An assertion in Playwright is a built-in feature used to verify that the actual result or behavior of an application matches the expected result. It helps confirm that the application is working as expected, and the test passes or fails accordingly.

withoutAssertion:

```
import { test } from '@playwright/test';

test('verify page title without assertion', async ({ page }) => {
  await page.goto('demowebshop.tricentis.com');

  const actualTitle = await page.title();
  const expectedTitle = 'Demo Web Shop';

  if (actualTitle === expectedTitle) {
    console.log('PASS: Title matches the expected title.');
  } else {
    console.log(`FAIL: Expected "${expectedTitle}" but got
    "${actualTitle}"`);
  }
});
```

pass → HTML-Report =pass
In Terminal=print pass

fail → HTML-Report =pass
In Terminal=print fail

Note:An if/else statement does not have the capability of failing the test case, even though the actual result is not matching with the expected result. That is the reason we should go with assertion.

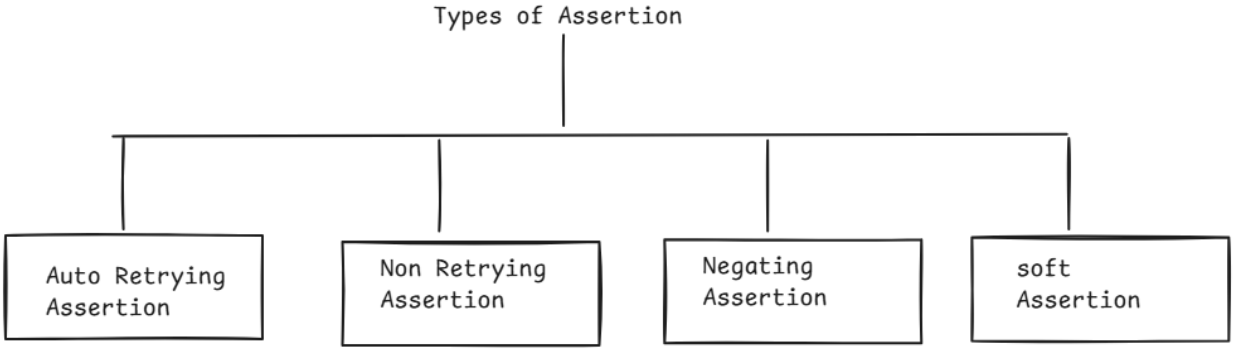
Usage of Assertion:
confirm UI state: element is visible, hidden, enabled, disabled, or checked.
URL: page navigated to the correct address.
Element behavior: element shows the correct text or value.
Element attributes: element has the expected attribute.
Other conditions: page title, element count, etc.

Arguments assertion will take: [Page/Value/Locater,{message}]

Synatx:

```
await expect(value/locator/page).assertionmethod(expected);
```

expect():expect() is a Playwright function used to verify whether the actual result matches the expected result



1)Auto-Retrying Assertion:[has Auto wait Mechansim for 5 sec]
Auto-retrying assertion: An auto-retrying assertion in Playwright is a built-in assertion that automatically checks a specified condition again and again until it becomes true or the timeout is reached. By default, Playwright waits up to 5 seconds for it to pass, and this timeout can be increased.

syntax: `await expect(loator/page).assertionmethod(expected)`

Methods

Method	Purpose
toBeVisible()	Checks whether the element is visible
toBeHidden()	Checks whether the element is hidden
toBeAttached()	Checks whether the element is attached to the DOM
toBeEnabled()	Checks whether the element is enabled
toBeDisabled()	Checks whether the element is disabled
toBeChecked()	Checks whether a checkbox or radio button is checked
toBeFocused()	Checks whether the element is focused
toHaveText()	Checks the element's text
toContainText()	Checks whether the element contains specific text
toHaveValue()	Checks the input field's value
toHaveAttribute()	Checks an attribute's value
toHaveClass()	Checks the element's class
toHaveCount()	Checks the number of matching elements
toHaveCSS()	Checks a CSS property
toHaveTitle()	Checks the page title
toHaveURL()	Checks the page URL
toHaveScreenshot()	Checks the screenshot against a baseline

```
test.only("autoRetryingAssertion",async({page})=>{
await page.goto("https://en.wikipedia.org/wiki/Main_Page")

//toHaveURL= !only excat match --> not partial match
await expect(page).toHaveURL("https://en.wikipedia.org/wiki/Main_Page")

//toHaveTitle= !only exact title ---> not partial title
await expect(page).toHaveTitle("Wikipedia, the free encyclopedia",{timeout:2000})

//tobevisible()= IT WILL CHECK WHETHER THE ELEMENT IS VISIBLE OR NOT
let textfield= page.locator("[placeholder='Search Wikipedia']").first()
expect(textfield).toBeVisible()

//! if u want to print the value which is present in the input field use ---> inputValue()
//! if u want to validate the value present in the input field use --> toHaveValue()
//toHaveValue() accepts an argument representing the expected value present in the input field.
await textfield.fill("mobile")
await expect(textfield).toHaveValue("hello")

//tocontainText =//! it will check for partial/ substring visible text
await expect(page.locator("//a[text()='Wikipedia']")).toContainText("Wiki")

//tohavetext()=  //! it will check for exact text / string visible text
await expect(page.locator("//a[text()='Wikipedia']")).toHaveText("Wikipedia")

//tohaveScreenshot()=whether the actual screenshot matches the expected screenshot.
await expect(page).toHaveScreenshot("wiki.png")
})
```

2)Non Retrying Assertion[not having auto-wait mechanism and don't have default timeout
Non-retrying assertion: A built-in assertion in Playwright that checks the condition only once and does not wait or retry if the condition is not true. Playwright evaluates the value immediately and gives the result: if the condition is true, the test passes, and if it is false, the test fails instantly.
These assertions are used when you want to verify values that are already available immediately, such as numbers, strings, boolean values, or variables.

syntax: `expect(value).assertionmethod(expected)`

methods

toBe()	1. Checks exact equality between two values.
toEqual()	2. Checks equality of values or objects.
toStrictEqual()	3. Checks strict equality of objects, including properties and data types.
toContain()	4. Checks whether an array or string contains a specified value.
toMatch()	5. Checks whether a string matches a specified pattern.
toBeTruthy()	6. Checks whether a value is truthy.
toBeFalsy()	7. Checks whether a value is falsy.
toBeNull()	8. Checks whether a value is `null`.
toBeUndefined()	9. Checks whether a value is `undefined`.
toBeGreaterThan()	10. Checks whether a value is greater than another value.
toBeLessThan()	11. Checks whether a value is less than another value.
toBeGreaterThanOrEqual()	12. Checks whether a value is greater than or equal to another value.
toBeLessThanOrEqual()	13. Checks whether a value is less than or equal to another value.

```
test.only("non retrying assertion",async({page})=>{
  await page.goto('https://demoapps.qspiders.com/ui?scenario=1')

  // toBe() = only exact match --> not partial match (case-sensitive, spaces also matter)
  let title=await page.title()
  expect(title).toBe("Demoapps | Qspider")

  let url=page.url()
 expect(url).toBe("https://demoapps.qspiders.com/ui?scenario=1")

  let text=await page.locator('//h1[.="Register"]').textContent()
  expect(text).toBe("Register")

// toMatch() = partial match using a regular expression --> passes if "demoapps" appears anywhere in the URL
  expect(url).toMatch(/demoapps/)

  // toContain() =checks if the expected text is present in the actual string, passes if "demoapps" appears anywhere in the URL
  expect(url).toContain("demoapps")

  // toBeGreaterThan() = checks if the actual number is greater than the expected number passes if the number of links is greater than 10
  const links = await page.locator('a').count();
  expect(links).toBeGreaterThan(10);

  //checks if the actual value is truthy (not empty, false, 0, null, or undefined)
  let status="hii"
  expect(status).toBeTruthy()

  //checks if the actual value is undefined
  let name="rohit";
   expect(name).toBeUndefined()
})
```

3)Negating Assertion

Negating assertions are used to verify that something is not true or does not exist. In simple words, they check the opposite condition: instead of checking that something is present, visible, or equal, they check that it is not present, not visible, or not equal.

In Playwright, negating assertions are written using .not with the expect() function, and the keyword .not tells Playwright to verify the opposite condition.They work with both auto-retrying and non-retrying assertions.

For example, if toBeVisible() checks that an element is visible,
then .not.toBeVisible() checks that the element is not visible.

syntax: `await expect(value/loator/page).not.assertionmethod(expected)`

```
Run | Debug | Inspect
86  test("negative assertion",async({page})=>{
87  await page.goto("https://demoapps.qspiders.com/ui?scenario=1")
88
89  //the test passes only when the condition is false.
90  //.not.toHaveURL() = checks that the actual URL does NOT match the expected URL
91  await expect(page).not.toHaveURL("https://demoapps.qspiders.com/ui?scenario=")
92
93  //.not.toBe() = checks if the actual value is NOT equal to the expected value
94  let title=await page.title()
95  expect(title).not.toBe("Demoapps | Qspider")
96  })
97
```

4)SoftAssertion

A soft assertion does not stop the test when it fails. Playwright records the failure, continues with the remaining steps, and marks the test as failed at the end, showing all failures together.

This is useful when you want to verify multiple things in a single test and see all the failures together instead of stopping at the first failure.

In Playwright, soft assertions are written using expect.soft() instead of expect().

syntax: `await expect.soft(value/loator/page).assertionmethod(expected)`

```
Run | Debug | Inspect
test("soft assertion",async({page})=>{
await page.goto("https://demoapps.qspiders.com/ui?scenario=1")
await expect.soft(page).toHaveURL("https://demoapps.qspiders.com/ui?scenario=")
let username:Locator= page.getByPlaceholder("Enter your name",{exact:true});
await expect.soft(username).toBeVisible({timeout:5000})
await expect.soft(page).toHaveTitle("DemoApps | Qspiders | Text Box",{timeout:2000})
})
```

Assertion with custom message

Custom messages are used to display your own message when an assertion fails, helping you understand the reason for the failure clearly.



```
Run | Debug | Inspect
test('Example of custom assertion messages', async ({ page }) => {
  await page.goto('https://demoapps.qspiders.com/ui?scenario=1');

  // Assertion with custom message for title
   await expect(page,'Page title is incorrect. Expected title should be Example Domain').toHaveTitle('DemoApps | Qspiders | Text Box');

  let username:Locator= page.getByPlaceholder("Enter your name",{exact:true});
 //Assertion with custom message for visibility
  await expect(username,'username element is not visible on the page').toBeVisible();

  const url = page.url();
  // Non-retrying assertion with custom message
  expect(url,'URL does not match the expected value').toBe('https://example.com/');

});
```